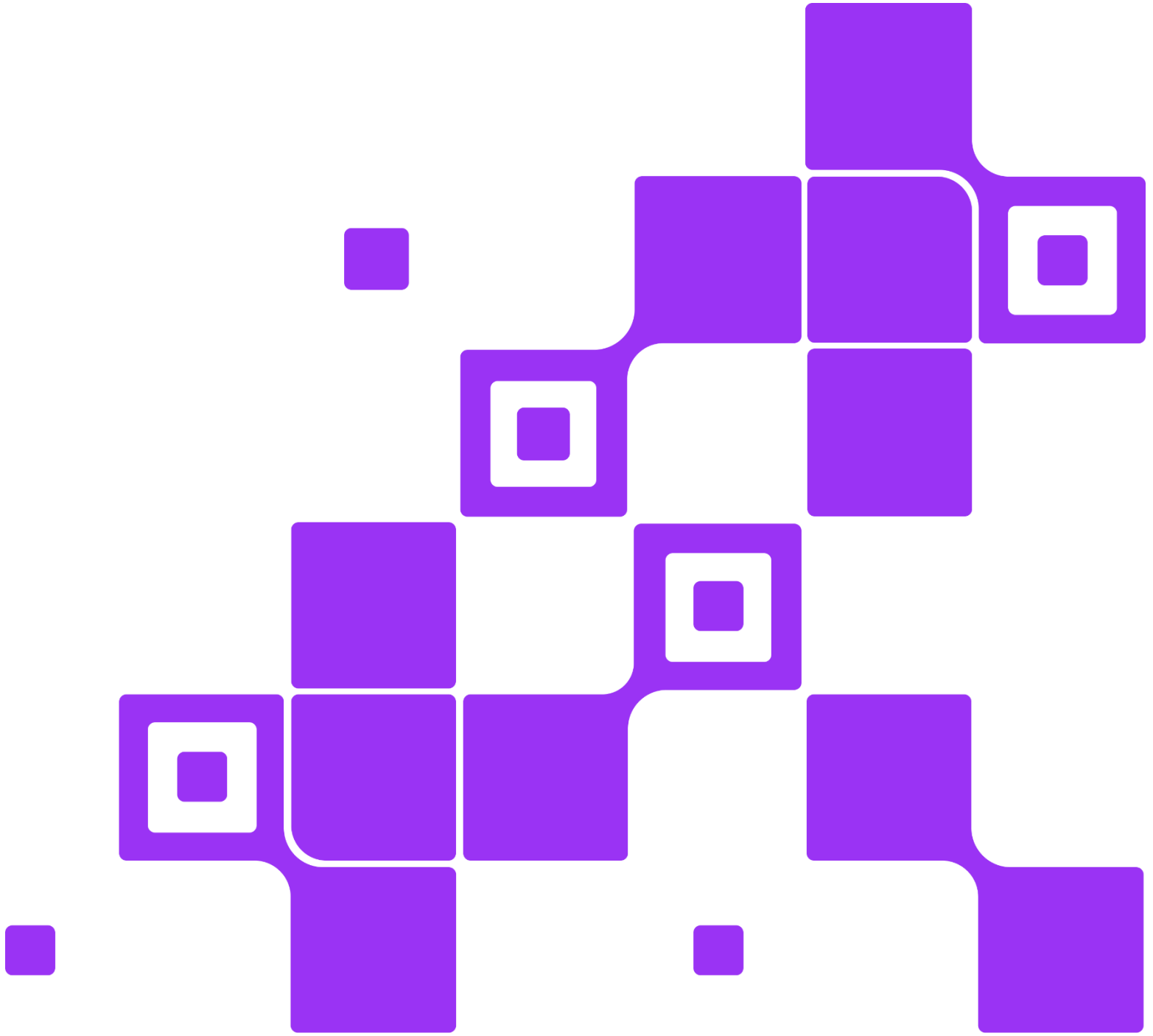
Интерфейс Map



IThub

Интерфейс Map

```
interface Map<K, V>
```

- интерфейс Map сопоставляет уникальные ключи со значениями. **Ключ** - это объект, который используется для извлечения значения в будущем. Имея ключ и значение, это значение можно сохранить в объекте Map. После сохранения получить значение можно с помощью ключа
- в K указывается тип ключей, а в V - тип значений

Полезные методы

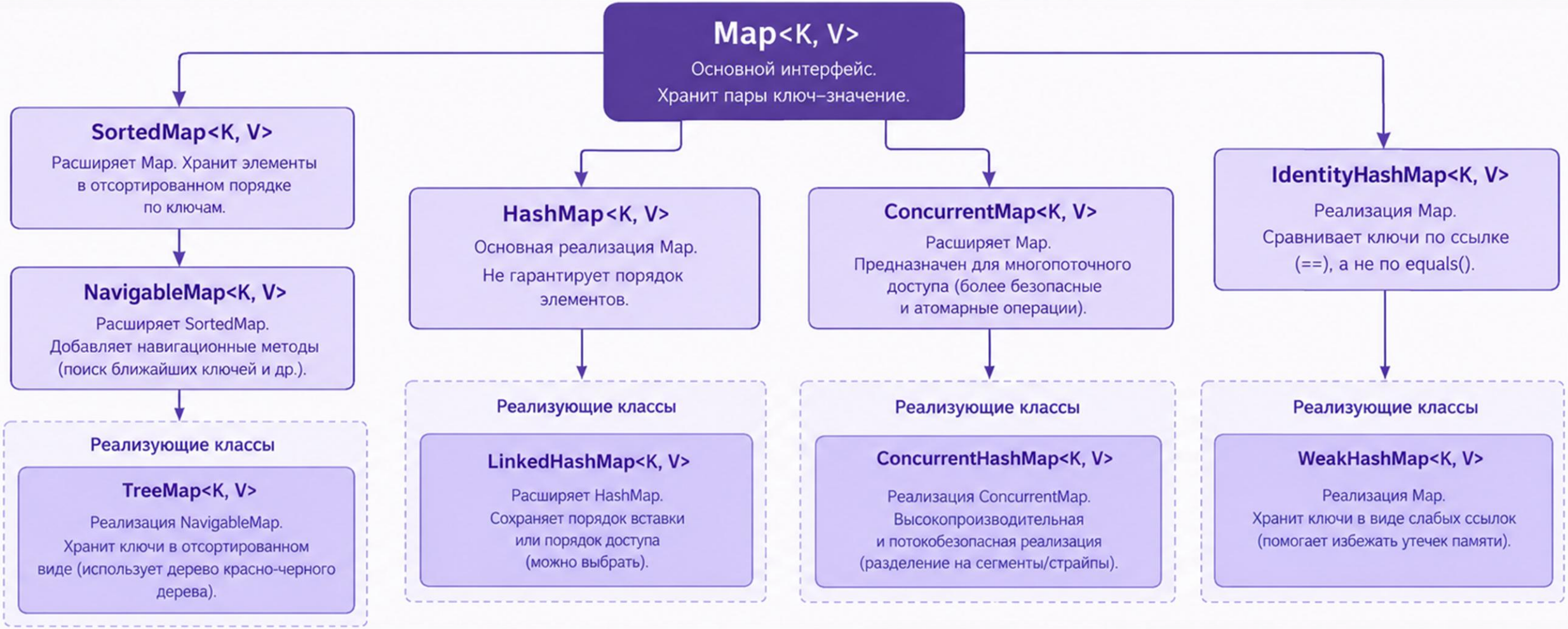
IThub

Метод	Что делает
V put(key, value)	добавляет/заменяет значение
V get(key)	получает значение по ключу
V remove(key)	удаляет элемент по ключу
boolean containsKey(key)	проверяет наличие ключа
boolean containsValue(value)	проверяет наличие значения
int size()	количество элементов
boolean isEmpty()	проверяет, пустая ли Map
void clear()	очищает Map
Set<K> keySet()	возвращает множество ключей
Collection<V> values()	возвращает коллекцию значений
Set<Map.Entry<K,V>> entrySet()	возвращает пары key-value
V getOrDefault(key, defaultValue)	возвращает значение или значение по умолчанию

Полезные методы

Метод	Что делает
V putIfAbsent(K key, V value)	добавляет значение, если ключ отсутствует
V computeIfAbsent(K key, Function...)	вычисляет значение, если ключ отсутствует
V computeIfPresent(K key, BiFunction...)	изменяет значение, если ключ существует
V compute(K key, BiFunction...)	пересчитывает значение
V merge(K key, V value, BiFunction...)	объединяет новое и существующее значение
void forEach(BiConsumer...)	выполняет действие для каждой пары

IThub


Map<K, V>
Основной интерфейс.
Хранит пары ключ–значение.
SortedMap<K, V>
Расширяет Map. Хранит элементы в отсортированном порядке по ключам.
NavigableMap<K, V>
Расширяет SortedMap.
Добавляет навигационные методы (поиск ближайших ключей и др.).
Реализующие классы
TreeMap<K, V>
Реализация NavigableMap.
Хранит ключи в отсортированном виде (использует дерево красно-черного дерева).
HashMap<K, V>
Основная реализация Map.
Не гарантирует порядок элементов.
Реализующие классы
LinkedHashMap<K, V>
Расширяет HashMap.
Сохраняет порядок вставки или порядок доступа (можно выбрать).
ConcurrentMap<K, V>
Расширяет Map.
Предназначен для многопоточного доступа (более безопасные и атомарные операции).
Реализующие классы
ConcurrentHashMap<K, V>
Реализация ConcurrentMap.
Высокопроизводительная и потокобезопасная реализация (разделение на сегменты/страйпы).
IdentityHashMap<K, V>
Реализация Map.
Сравнивает ключи по ссылке (==), а не по equals().
Реализующие классы
WeakHashMap<K, V>
Реализация Map.
Хранит ключи в виде слабых ссылок (помогает избежать утечек памяти).

SortedMap и NavigableMap

IThub

```
interface SortedMap<K,V> extends Map<K,V>
```

это Map, в которой ключи всегда находятся в отсортированном порядке.

```
interface NavigableMap<K,V> extends SortedMap<K,V>
```

NavigableMap имеет все методы SortedMap + дополнительные методы навигации по ключам.

Класс TreeMap

```
class TreeMap<K, V> extends AbstractMap<K, V> implements NavigableMap<K, V>
```

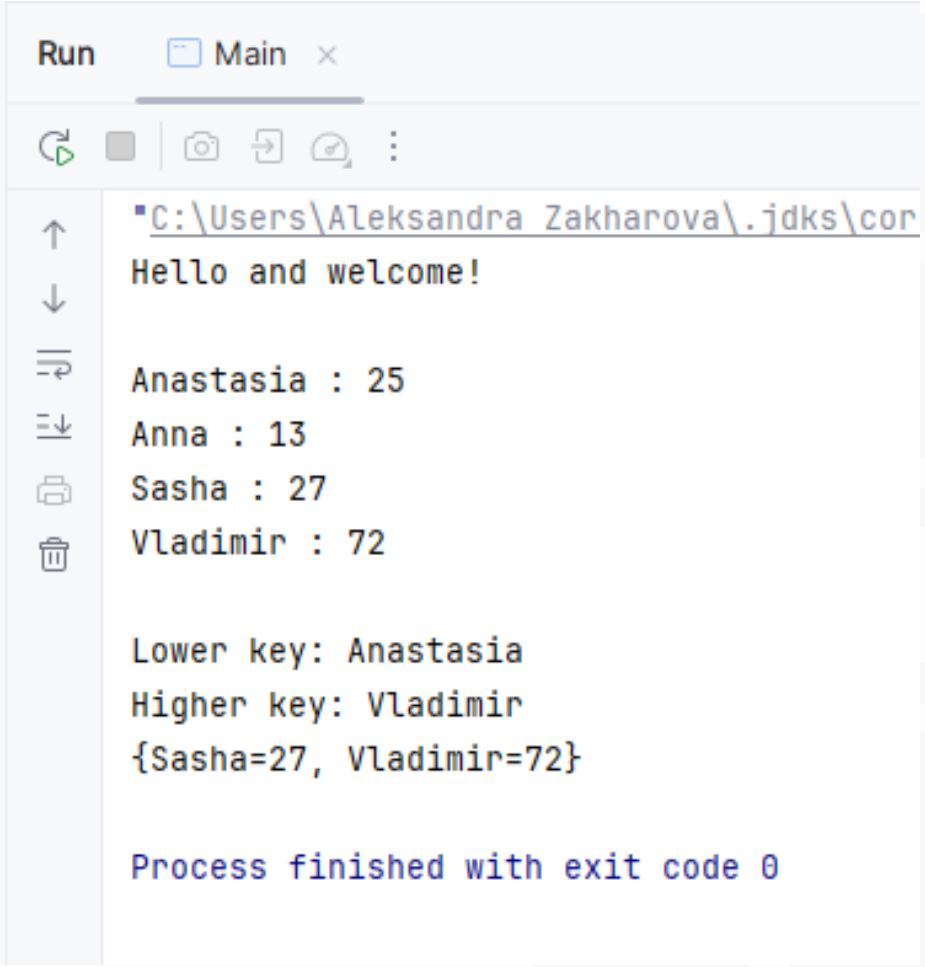
- для хранения карты используется древовидная структура
- хранение пар "ключ-значение" осуществляется в отсортированном порядке и обеспечивает быстрое извлечение
- гарантирует, что ее элементы будут отсортированы в порядке возрастания ключей

! эта реализация **не синхронизирована** (если несколько потоков обращаются к карте одновременно, и хотя бы один из них изменяет её структуру, необходимо выполнить внешнюю синхронизацию)

```java
public void treeMapDemo() {
    TreeMap<String, Integer> treeMap = new TreeMap<>();
    treeMap.put("Sasha", 27);
    treeMap.put("Anastasia", 25);
    treeMap.put("Anna" , 13);
    treeMap.put("Vladimir" , 72);

    for (Map.Entry<String, Integer> entry : treeMap.entrySet()) {
        System.out.println(entry.getKey() + " : " + entry.getValue());
    }

    System.out.println("\nLower key: " + treeMap.firstEntry().getKey());
    System.out.println("Higher key: " + treeMap.lastEntry().getKey());
    System.out.println(treeMap.tailMap("Sasha"));
}
```



firstEntry()	Получить первую пару key-value
lastEntry()	Получить последнюю пару key-value
tailMap(...)	Получить ключи от заданного

Класс HashMap

IThub

```
class HashMap<K, V> extends AbstractMap<K, V> implements Map<K, V>
```

- для хранения карты применяется хеш-таблица
- хеш-карта не гарантирует порядок следования элементов

! эта реализация не синхронизирована (если несколько потоков одновременно обращаются к хеш-карте, и хотя бы один из них изменяет её структуру, необходимо выполнить внешнюю синхронизацию)

Хэширование

Хэширование — это способ быстро преобразовать объект (например, ключ) в некоторое числовое значение — хэш-код.

В Java для этого используется метод hashCode() (от Object).

```
map.put("Alice", 25);
```

HashMap примерно делает следующее:

"Alice"
↓ вычисляется hash
↓ определяется bucket
↓ сохраняется ключ + значение

Что такое Bucket?

Bucket — это ячейка внутреннего массива Node<K,V>[] table в HashMap, которая хранит:

- либо null (пустой бакет);
- либо ссылку на первый узел связного списка LinkedList (или красно-чёрного дерева TreeMap, если элементов > 8).

Один bucket может иметь более, чем один узел, это зависит от реализации метода hashCode().

Как определяется номер бакета?

IThub

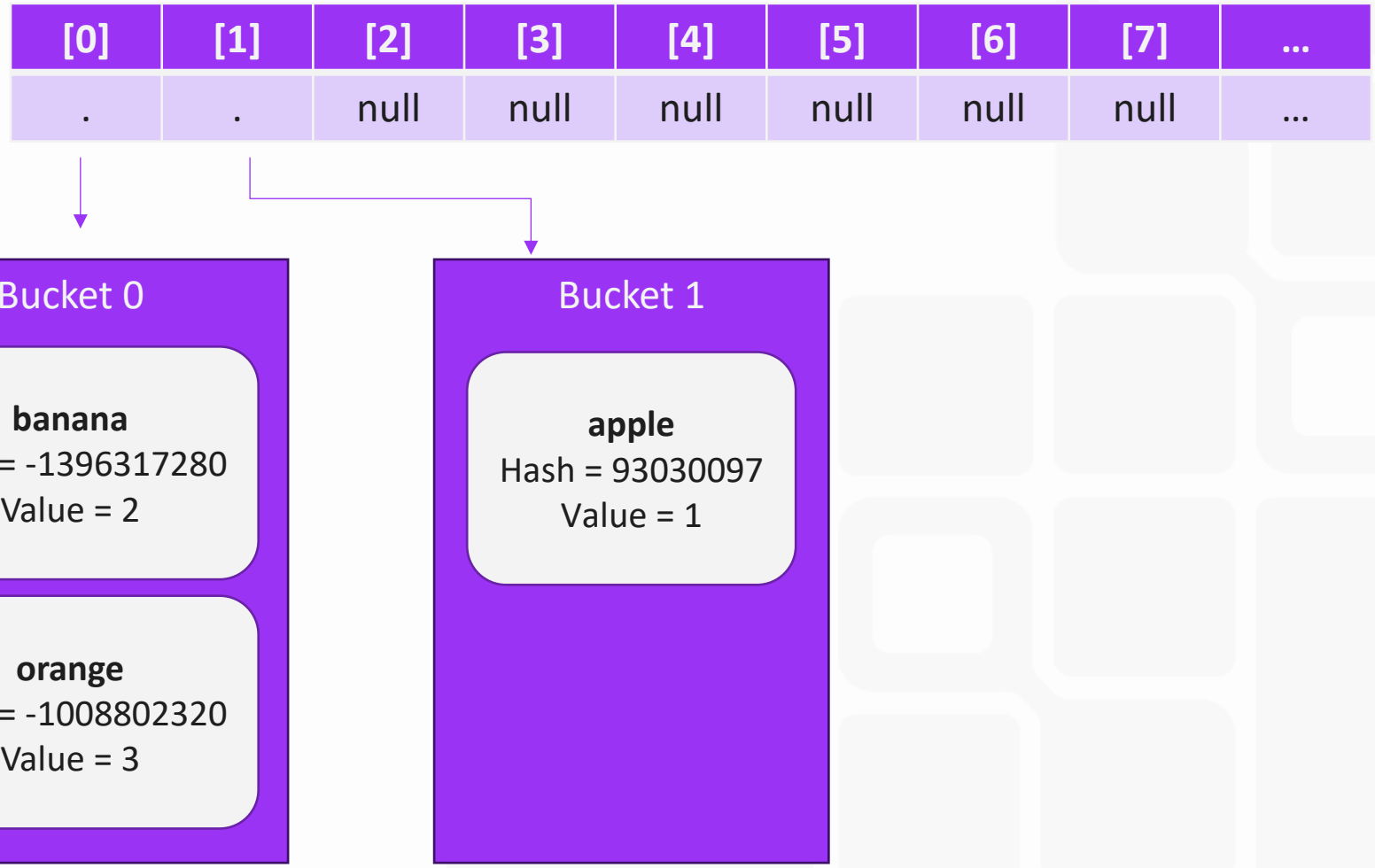
```
int hashCode = key.hashCode();        // хеш ключа
int hash = hashCode ^ (hashCode >>> 16)
int index = (n - 1) & hash;           // n = длина массива (16 по умолчанию)
```

Пример:

```
HashMap<String, Integer> map = new HashMap<>();
map.put("apple", 1);
map.put("banana", 2);
map.put("orange", 3);
```

Ключ	hashCode()	hash = h ^ (h >>> 16)	index = (16-1) & hash	Бакет
"apple"	93029210	93030097	93030097 & 15 = **1**	bucket[1]
"banana"	-1396355227	-1396317280	-1396317280 & 15 = **0**	bucket[0]
"orange"	-1008851410	-1008802320	-1008802320 & 15 = **0**	bucket[0] **коллизия!**

```
HashMap<String, Integer> map = new HashMap<>();
map.put("apple", 1);
map.put("banana", 2);
map.put("orange", 3);
```



Связать новый и старый объекты, указав ссылку на следующий объект в текущем

Поиск — сначала по бакету (O(1)), затем внутри бакета (O(n) или O(log n))

Rehashing

IThub

Rehashing (перехеширование) — это процесс увеличения внутреннего массива table и перераспределения всех элементов по новым бакетам, когда HashMap заполняется.

Проблема возникает, когда элементов становится много: бакеты начинают переполняться, коллизий становится больше, а операции поиска могут замедляться.

Главный параметр, который определяет, когда пора увеличивать HashMap — **load factor** (по-дефолту = 0,75).

Тогда порог **threshold** = capacity × loadFactor = 16 × 0.75 = 12

Rehashing

IThub

Допустим, было: capacity = 16 load factor = 0.75 threshold = 12

И мы добавили 13-й элемент. HashMap понимает, что таблица заполнена сильнее допустимого и нужно увеличить её.

Размер обычно удваивается: 16 → 32

Но просто скопировать элементы на те же индексы нельзя. Нужно перераспределить элементы по новой таблице.

index = hash & (capacity - 1)

- При capacity = 16

index = hash & 15

- После увеличения (capacity = 32)

index = hash & 31

Поэтому один и тот же hash может дать другой индекс.

Rehashing

IThub

В современных реализациях Java HashMap есть очень полезная оптимизация.

Когда capacity увеличивается в 2 раза, элемент из старой корзины может:

- либо остаться на том же индексе
- либо переместиться на oldCapacity вперёд

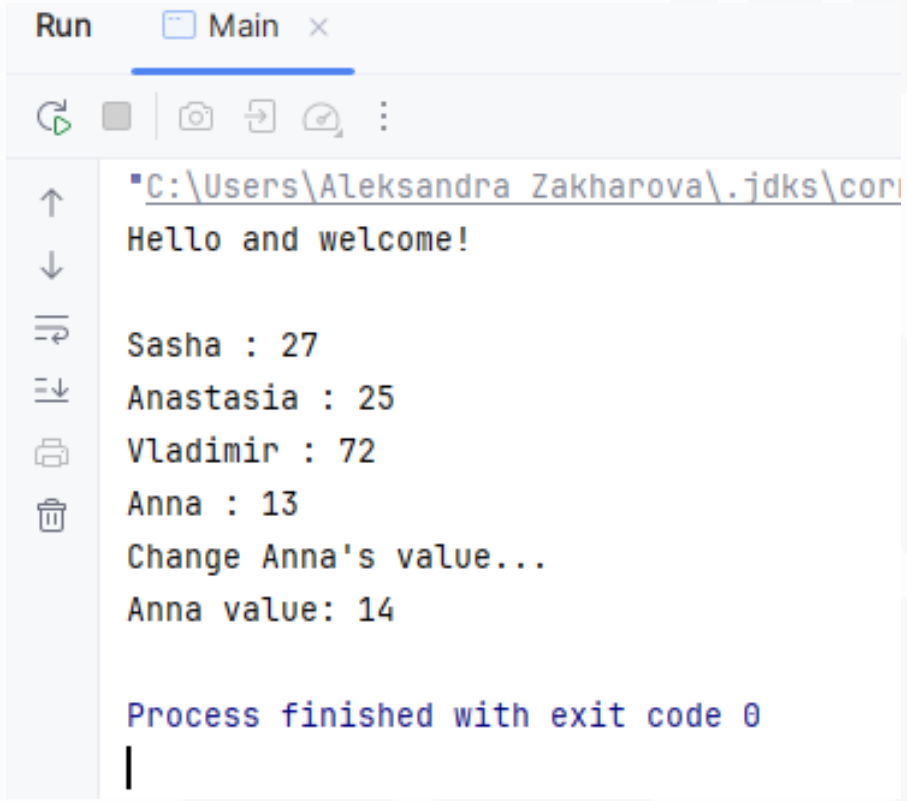
Класс HashMap

IThub

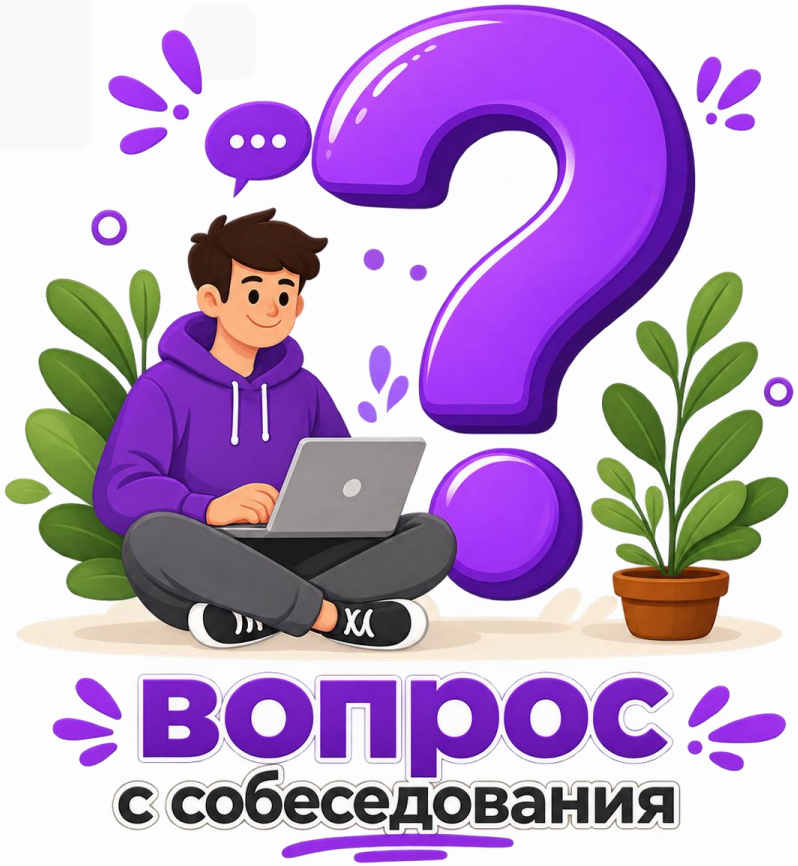
```
public void hashMapDemo() {
    HashMap<String, Integer> hashMap = new HashMap<>();
    hashMap.put("Sasha", 27);
    hashMap.put("Anastasia", 25);
    hashMap.put("Anna" , 13);
    hashMap.put("Vladimir" , 72);

    for (Map.Entry<String, Integer> entry : hashMap.entrySet()) {
        System.out.println(entry.getKey() + " : " + entry.getValue());
    }

    System.out.println("Change Anna's value...");
    hashMap.put("Anna" , 14);
    System.out.println("Anna value: " + hashMap.get("Anna"));
}
```



IThub



Почему строка (тип String) является популярным ключом в HashMap в Java?

Поскольку строки **неизменяемы**, их хэш код вычисляется и кэшируется в момент создания, не требуя повторного пересчета при дальнейшем использовании. Поэтому в качестве ключа HashMap они будут обрабатываться быстрее.

Класс LinkedHashMap

```
class LinkedHashMap<K, V> extends HashMap<K, V> implements Map<K, V>
```

- поддерживает связный список элементов карты в том порядке, в котором они были вставлены. Это позволяет выполнять итерацию по карте в порядке вставки

! эта реализация **не синхронизирована** (если несколько потоков одновременно обращаются к связанной хеш-карте, и хотя бы один из них изменяет её структуру, необходимо выполнить внешнюю синхронизацию)

Класс LinkedHashMap

```
public void linkedHashMapDemo() {
    LinkedHashMap<Integer, String> linkedHashMap = new LinkedHashMap<>();
    linkedHashMap.put(27, "Sasha");
    linkedHashMap.put(14, "Anna");
    linkedHashMap.put(72, "Vladimir");
    linkedHashMap.put(55, "Anastasia");

    System.out.println("LinkedHashMap  = " + linkedHashMap);

    System.out.println("Contains value Sasha = " + linkedHashMap.containsValue("Sasha"));
    System.out.println("Contains key 23 = " + linkedHashMap.containsKey(23));
    System.out.println("Key set = " + linkedHashMap.keySet());
}
```

```
"C:\Users\Aleksandra Zakharova\.jdks\corretto-17.0.14\bin\java.exe
Hello and welcome!

LinkedHashMap  = {27=Sasha, 14=Anna, 72=Vladimir, 55=Anastasia}
Contains value Sasha = true
Contains key 23 = false
Key set = [27, 14, 72, 55]

Process finished with exit code 0
```

Класс IdentityHashMap

```
class IdentityHash<K, V> extends AbstractMap<K,V> implements Map<K,V>
```

- этот класс не является реализацией Map общего назначения!
- Хотя этот класс реализует интерфейс Map, он намеренно нарушает общий контракт Map, который предписывает использование метода equals при сравнении объектов.
- Этот класс предназначен для использования только в редких случаях, когда **требуется семантика равенства ссылок**

! эта реализация **не синхронизирована**

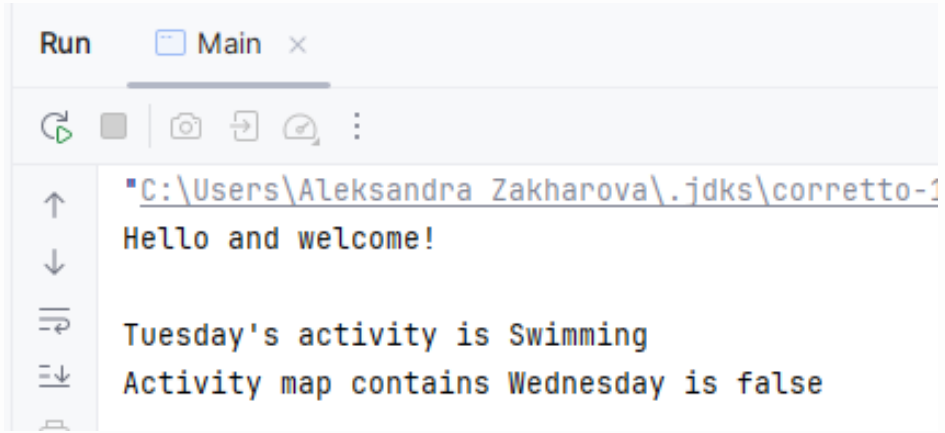
Класс EnumMap

```
class EnumMap<K extends Enum<K>,V> extends AbstractMap<K,V>
```

- специализированная Map реализация для использования с ключами типа enum.
- Карты enum представляются внутри как массивы. Такое представление чрезвычайно компактно и эффективно.

Класс EnumMap

```
public enum DayOfWeek {
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY;
}
```




```
public void enumMapDemo() {
    EnumMap<DayOfWeek, String> activityMap = new EnumMap<>(DayOfWeek.class);
    activityMap.put(DayOfWeek.MONDAY, "Football");
    activityMap.put(DayOfWeek.TUESDAY, "Swimming");
    activityMap.put(DayOfWeek.FRIDAY, "Basketball");

    System.out.println("Tuesday's activity is " + activityMap.get(DayOfWeek.TUESDAY));
    System.out.println("Activity map contains Wednesday is " + activityMap.containsKey(DayOfWeek.WEDNESDAY));
}
```

IThub



ArrayList
Список

- Быстрый доступ по индексу
- Хорош для чтения
- Изменения в середине/начале — неэффективны

Когда использовать:
- Частый доступ по индексу
- Большое количество чтений

Пример:

```
List<String> list =
      new ArrayList<>();
```

LinkedList
Список

- Быстрая вставка/удаление в начале и конце
- Нет быстрого доступа по индексу

Когда использовать:
- Частые вставки/удаления
- Итераторы, очереди, стеки

Пример:

```
Deque<String> deque =
      new LinkedList<>();
```

HashSet
Набор

- Уникальные элементы
- Высокая производительность
- Порядок не гарантирован

Когда использовать:
- Нужно быстро проверять наличие элемента
- Порядок не важен

Пример:

```
Set<String> set =
      new HashSet<>();
```

LinkedHashSet
Набор

- Уникальные элементы
- Сохраняет порядок вставки
- Чуть медленнее, чем HashSet

Когда использовать:
- Нужен порядок элементов
- И при этом важна уникальность

Пример:

```
Set<String> set =
      new LinkedHashSet<>();
```

HashMap
Карта

- Быстрый доступ по ключу
- Не гарантирует порядок
- Разрешает null (ключ и значение)

Когда использовать:
- Быстрый поиск по ключу
- Порядок не важен

Пример:

```
Map<String, Integer> map =
      new HashMap<>();
```

TreeMap
Карта

- Ключи отсортированы
- Есть навигация по ключам
- Не разрешает null (ключ)

Когда использовать:
- Нужна сортировка ключей
- Нужна навигация (firstKey, floorKey и т.д.)

Пример:

```
Map<Integer, String> map =
      new TreeMap<>();
```

Задача (HashMap)

IThub

Разработайте программу для анализа банковских транзакций.

HashMap<String, Integer> хранит **имя_клиента : сумма**

- Положительное число— пополнение счёта
- Отрицательное число — списание денег

Задание:

1. Собрать текущий баланс каждого клиента
2. Вывести клиентов и их баланс
3. Найти клиента с максимальным балансом
4. Посчитать, сколько клиентов имеют отрицательный баланс
5. Выполнить операцию для клиента (положить деньги на баланс, снять деньги с баланса)
6. Найти самого активного клиента
7. Добавить поиск по имени
    - Если найден – вывести баланс и число операций
    - Если не найден – вывести «Клиент не найден»